



Chair of
Information Theory
and Data Analytics

RWTHAACHEN
UNIVERSITY

RESOURCE-EFFICIENT INFERENCE IN LARGE GENERATIVE MODELS

PATRICK TAZO KUETE

CHAIR OF INFORMATION THEORY AND DATA ANALYTICS
RWTH AACHEN UNIVERSITY



Chair of
Information Theory
and Data Analytics

RWTHAACHEN
UNIVERSITY

RESOURCE-EFFICIENT INFERENCE IN LARGE GENERATIVE MODELS

PATRICK TAZO KUETE

A seminar submitted to the
Faculty of Electrical Engineering and Information Technology,
RWTH Aachen University,

reviewed by Univ.-Prof. Dr. Ing. Anke Schmeink and supervised by Hazem el Hadj Khalifa,
M.Sc.

CHAIR OF INFORMATION THEORY AND DATA ANALYTICS
RWTH AACHEN UNIVERSITY

Eidesstattliche Versicherung

Name, Vorname

Matrikelnummer

Ich versichere hiermit an Eides Statt, dass ich die vorliegende Seminararbeit mit dem Titel

Resource-Efficient Inference in Large Generative Models

selbständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt. Für den Fall, dass die Arbeit zusätzlich auf einem Datenträger eingereicht wird, erkläre ich, dass die schriftliche und die elektronische Form vollständig übereinstimmen. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Aachen, den 18th June 2026

Ort, Datum

Unterschrift

Belehrung:

§ 156 StGB: Falsche Versicherung an Eides Statt

Wer vor einer zur Abnahme einer Versicherung an Eides Statt zuständigen Behörde eine solche Versicherung falsch abgibt oder unter Berufung auf eine solche Versicherung falsch aussagt, wird mit Freiheitsstrafe bis zu drei Jahren oder mit Geldstrafe bestraft.

§ 161 StGB: Fahrlässiger Falscheid; fahrlässige falsche Versicherung an Eides Statt

(1) Wenn eine der in den §§ 154 bis 156 bezeichneten Handlungen aus Fahrlässigkeit begangen worden ist, so tritt Freiheitsstrafe bis zu einem Jahr oder Geldstrafe ein.

(2) Strafflosigkeit tritt ein, wenn der Täter die falsche Angabe rechtzeitig berichtigt. Die Vorschriften des § 158 Abs. 2 und 3 gelten entsprechend

Die vorstehende Belehrung habe ich zur Kenntnis genommen:

Aachen, den 18th June 2026

Ort, Datum

Unterschrift

Abstract

Large generative models, in particular decoder-only Transformer architectures, have demonstrated unprecedented capabilities in natural language processing, code synthesis, and multimodal reasoning. However, the inference process is computationally intensive: repeated autoregressive forward passes impose severe demands on memory, compute, memory bandwidth, and energy. These characteristics make deployment on resource-constrained edge devices — such as smartphones, embedded systems, and Internet of Things (IoT) hardware — a significant open challenge.

This paper provides a structured survey of resource-efficient inference in large generative models. Starting from the fundamental Transformer building blocks, it traces the complete inference pipeline and systematically identifies the principal resource bottlenecks at each stage, distinguishing between the compute-bound prefill phase and the memory-bandwidth-bound decode phase. Building on this analysis, the paper surveys the leading efficiency techniques across four layers: model compression (quantisation, pruning, and knowledge distillation), inference-level algorithmic improvements (optimised attention mechanisms and Key-Value (KV) cache management), decoding strategies (speculative decoding and batching), and system-level runtimes (vLLM, llama.cpp, and MLC-LLM). For each technique, the discussion focuses on the specific bottleneck it addresses and the practical trade-offs that arise when targeting edge hardware.

The analysis shows that quantisation combined with grouped-query attention and speculative decoding currently represents the most promising combination for practical edge inference, enabling models with billions of parameters to run on consumer-grade hardware with acceptable quality degradation.

Contents

List of Abbreviations	xv
1 Introduction	1
1.1 Motivation and Objectives	1
1.2 Defining Resources in the Context of Inference	2
1.2.1 Compute	3
1.2.2 Memory	3
1.2.3 Memory Bandwidth	3
1.2.4 Energy	4
1.3 The Transformer Architecture	4
1.3.1 Tokenisation and Embedding	4
1.3.2 Transformer Blocks	4
1.3.3 Language Modelling Head	5
1.4 Key Terminology	5
1.5 Scope: Why Edge Devices?	6
2 The Inference Process and its Bottlenecks	9
2.1 Overview of the Inference Pipeline	9
2.2 Step-by-Step Bottleneck Analysis	10
2.2.1 Step 1: Tokenisation	10
2.2.2 Step 2: Token Embedding and Positional Encoding	11
2.2.3 Step 3: Multi-Head Self-Attention	11
2.2.4 Step 4: Feed-Forward Network	11
2.2.5 Step 5: LM Head and Sampling	12
2.3 Limitations Beyond Inference	12
3 Towards Efficient Inference: Techniques and Solutions	15
3.1 Model Compression	16
3.1.1 Quantisation	16
3.1.2 Pruning	17
3.1.3 Knowledge Distillation	18
3.2 Inference-Level Optimisations	19
3.2.1 Multi-Query and Grouped-Query Attention	19
3.2.2 FlashAttention	20
3.2.3 KV Cache Optimisation	20

3.3	Decoding Strategies	21
3.3.1	Speculative Decoding	21
3.3.2	Continuous Batching	22
3.4	System-Level Runtimes and Edge Frameworks	22
3.4.1	llama.cpp	22
3.4.2	MLC-LLM	23
3.4.3	vLLM	23
3.4.4	Comparison and Edge Suitability	23
3.5	Discussion: Trade-offs and Edge-Specific Considerations	24
3.5.1	The Efficiency–Accuracy Trade-off	24
3.5.2	Orthogonality and Composability	24
3.5.3	The Fundamental Edge Constraint: Memory	25
4	Conclusion	27
4.1	Key Findings	27
4.2	Trade-off Summary	28
4.3	Future Directions	29
	Bibliography	31

List of Figures

- 2.1 Schematic of the Transformer inference pipeline. Red-shaded steps (3, 4) represent the primary resource bottlenecks. The decode loop repeats steps 3–5 until an end-of-sequence token is produced. 10
- 3.1 Mapping of inference bottlenecks (red) to the efficiency techniques (blue) surveyed in this chapter. Arrows indicate which technique targets which bottleneck; several techniques address multiple bottlenecks simultaneously. . 15

List of Tables

2.1	Summary of inference bottlenecks by phase. n = sequence length, P = total parameters, L = layers, BW = memory bandwidth.	13
3.1	Comparison of selected inference frameworks for edge deployment. ✓ = supported, ~ = partial support, × = not supported.	24

List of Abbreviations

AI Artificial Intelligence 1

DRAM Dynamic Random Access Memory 3, 4, 9, 11, 25, 27

FFN Feed-Forward Network 5, 9, 11, 13, 17, 27

FLOP Floating Point Operation 3, 17

FP16 16-bit Floating Point 3, 16, 17, 20, 27

GPU Graphics Processing Unit 2, 4, 20–23

GQA Grouped-Query Attention 12, 19, 20, 24, 25, 27–29

HBM High-Bandwidth Memory 2, 4, 20

INT4 4-bit Integer 16

INT8 8-bit Integer 16, 17, 21

IoT Internet of Things vii, 2

KD Knowledge Distillation 18

KV Key-Value vii, 3, 5, 6, 9–13, 18–21, 23–25, 27–29

LLM Large Language Model 1, 2, 4, 6, 15–17, 22, 27, 29

LN Layer Normalisation 4

- LoRA** Low-Rank Adaptation 18, 29
- MFU** Model FLOP Utilisation 3, 11
- MHA** Multi-Head Attention 5, 19, 20
- MLC** Machine Learning Compilation vii, 23
- MLP** Multilayer Perceptron 5
- MQA** Multi-Query Attention 12, 19, 20
- NPU** Neural Processing Unit 2, 6, 29
- PTQ** Post-Training Quantisation 16, 17
- QAT** Quantisation-Aware Training 17
- RoPE** Rotary Positional Encoding 4
- SRAM** Static Random Access Memory 2, 4, 20

1 Introduction

The past decade has witnessed a remarkable transformation in Artificial Intelligence (AI) research, driven by the emergence and rapid scaling of large generative models. Systems such as GPT-3 [4], LLaMA [17], and their successors have demonstrated near-human performance on a broad spectrum of tasks—from natural language understanding and code generation to mathematical reasoning and multimodal dialogue. This progress rests fundamentally on the Transformer architecture [18], which, when trained on trillions of tokens and scaled to hundreds of billions of parameters, exhibits surprising emergent capabilities.

Yet capability has come at a steep cost. Generating even a single response with a state-of-the-art Large Language Model (LLM) requires billions of floating-point operations, gigabytes of memory, and significant electrical power. In data-centre settings this cost is absorbed through specialised hardware and economies of scale, but it nonetheless drives substantial economic and environmental expenditure. On resource-constrained platforms—smartphones, embedded controllers, autonomous vehicles—such demands can make deployment entirely infeasible without dedicated optimisation. The field of resource-efficient LLM inference has therefore emerged as one of the most active and practically consequential research areas in machine learning [3].

This paper provides a structured survey of the state of the art in resource-efficient inference for large generative models. It begins by grounding the analysis in a concrete understanding of what ‘resources’ means in this context (Section section 1.2), proceeds to a concise overview of the Transformer architecture and the terminology required for the remainder of the paper (Sections section 1.3–section 1.4), and introduces the scope and motivation for focusing specifically on edge devices (Section section 1.5). The detailed analysis of the inference pipeline and its bottlenecks is presented in Chapter chapter 2, and the comprehensive survey of efficiency techniques follows in Chapter chapter 3.

1.1 Motivation and Objectives

The economics of generative AI inference are fundamentally different from those of training. Training is a one-time cost, amenable to the use of large distributed compute clusters

over days or weeks. Inference, by contrast, must occur in real time for every user request—and the number of such requests is growing at an exponential rate as these models are embedded into products and workflows. A single 70 billion-parameter model served at modest throughput can require eight Graphics Processing Units (GPUs) equipped with High-Bandwidth Memory (HBM), each consuming 300–400 W. The aggregate cost and carbon footprint at societal scale is non-trivial [3].

Simultaneously, a growing set of applications demands inference at the *edge*—that is, on devices with severely constrained memory, compute, and energy budgets. These include, but are not limited to:

- **Mobile devices:** smartphones and tablets offer 12–16 GB of shared system memory, a modest Neural Processing Unit (NPU), and a battery-imposed power envelope of 3–8 W.
- **IoT hardware:** microcontrollers and system-on-chip platforms may have only megabytes of Static Random Access Memory (SRAM) and operate from energy-harvesting sources.
- **Embedded AI systems:** autonomous vehicles, industrial robots, and wearables require low-latency responses but cannot maintain a reliable network connection to a cloud backend.
- **Privacy-sensitive applications:** on-device processing avoids transmitting sensitive data—medical records, personal communications, financial transactions—to a remote server.

The objective of this paper is therefore two-fold: to provide a clear analytical framework for understanding *where* the resource bottlenecks in transformer inference arise, and to survey the techniques that address them, with particular emphasis on those that are relevant to edge deployment.

1.2 Defining Resources in the Context of Inference

The term ‘resource’ in the context of LLM inference encompasses four distinct but inter-related dimensions, each of which imposes a practical constraint on deployment [3].

1.2.1 Compute

Computational cost is measured in Floating Point Operations (FLOPs). A decoder-only model with P parameters requires approximately $2P$ FLOPs per forward pass per token during the decode phase, because each parameter participates in one multiply and one accumulate operation. For a 7-billion-parameter model this yields roughly 14 GFLOPs per generated token [15]. The primary metric for compute efficiency is Model FLOP Utilisation (MFU), which expresses what fraction of peak hardware FLOP throughput is actually utilised. In practice, MFU values below 50% are common during autoregressive decoding because the decode phase is memory-bandwidth-bound rather than compute-bound.

1.2.2 Memory

Memory constraints arise from two distinct sources:

Model weights. Each parameter stored in 16-bit floating point (16-bit Floating Point (FP16)) consumes 2 bytes. A 7B model therefore requires 14 GB of weight memory; a 70B model requires 140 GB. Current edge devices typically provide 8–16 GB of total system memory, meaning a 7B model at full precision barely fits a flagship smartphone and leaves almost no room for the activation tensors.

KV cache. During inference, the intermediate key and value tensors produced by the attention layers must be stored for every past token to avoid recomputation. The KV cache size scales as $2 \cdot L \cdot n \cdot d_{\text{kv}} \cdot \text{sizeof}(\text{dtype})$, where L is the number of layers, n is the current sequence length, and d_{kv} is the per-head dimension. For a 7B model with 32 layers, 32 heads, and a head dimension of 128 at FP16, a sequence of 2048 tokens requires approximately 8 GB of KV cache—matching the model weights themselves. This cache grows linearly with sequence length and can dominate memory usage for long-context tasks.

1.2.3 Memory Bandwidth

Even when sufficient memory is available, the speed at which data can move between storage (typically Dynamic Random Access Memory (DRAM)) and compute units determines throughput. During the decode phase, the model produces one token per forward pass. In this regime, all P weight parameters and the entire KV cache must be read from DRAM for each generated token, yet they contribute very little compute (because the input is a single token vector). The ratio of memory accesses to arithmetic operations is therefore extremely high: the operation is *memory-bandwidth-bound* [15]. High-bandwidth

memory (HBM) in GPU accelerators partially alleviates this; edge devices that rely on low-bandwidth DRAM or even SRAM face a more severe bottleneck.

1.2.4 Energy

Energy consumption is measured in joules per generated token. For edge devices it is the most fundamental constraint: a smartphone battery stores roughly 50 kJ, and a continuous inference workload at even 5 W drains the battery in a few hours. DRAM accesses are energetically expensive—approximately 200 pJ per 64-bit read, compared to 1 pJ for a floating-point multiply in an on-chip arithmetic unit. This makes reducing the volume of DRAM traffic (through caching, quantisation, or architectural changes) a prerequisite for energy-efficient edge inference.

1.3 The Transformer Architecture

Modern LLMs are built on the Transformer [18], specifically the *decoder-only* variant introduced by GPT-style architectures. Understanding the architecture at a functional level is necessary to reason about where bottlenecks arise.

1.3.1 Tokenisation and Embedding

Raw text is first converted into a sequence of integer token IDs by a tokeniser (e.g., Byte-Pair Encoding). Each token ID is then mapped to a dense real-valued vector $\mathbf{e} \in \mathbb{R}^{d_{\text{model}}}$ via a learned embedding matrix $\mathbf{W}_E \in \mathbb{R}^{|V| \times d_{\text{model}}}$, where $|V|$ is the vocabulary size (typically 32 000–128 000) and d_{model} is the model dimension (e.g., 4096 for a 7B model). A positional encoding—either fixed sinusoidal [18] or learned rotary (Rotary Positional Encoding (RoPE))—is added to inject information about token order.

1.3.2 Transformer Blocks

The sequence of d_{model} -dimensional vectors is then passed through L identical Transformer blocks (e.g., $L = 32$ for a 7B model). Each block contains two sub-layers connected by residual connections and layer normalisation (Layer Normalisation (LN)):

Multi-Head Attention (MHA). Given input $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$, the attention sub-layer projects $\mathbf{X}$ into h sets of query, key, and value matrices using learned weight matrices $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ for head i :

$$\text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i) = \text{softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{d_k}}\right) \mathbf{V}_i. \quad (1.3.1)$$

The h head outputs are concatenated and projected by $\mathbf{W}^O \in \mathbb{R}^{hd_k \times d_{\text{model}}}$. The causal mask in decoder-only models ensures that each token can only attend to itself and earlier tokens.

Feed-Forward Network (FFN). The feed-forward sub-layer is a two-layer Multilayer Perceptron (MLP) with a non-linear activation:

$$\text{FFN}(\mathbf{x}) = \sigma(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2, \quad (1.3.2)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, and $d_{\text{ff}} = 4d_{\text{model}}$ in the standard configuration. Modern architectures replace the ReLU activation with SwiGLU or GELU and use a gated variant with three weight matrices, slightly increasing parameter count. The FFN sublayer accounts for roughly two-thirds of a Transformer block’s parameters.

1.3.3 Language Modelling Head

After the final Transformer block, a linear projection $\mathbf{W}_{LM} \in \mathbb{R}^{d_{\text{model}} \times |V|}$ (often shared with $\mathbf{W}_E$) maps the last hidden state to a distribution over the vocabulary. A softmax followed by a sampling strategy (greedy, temperature sampling, top- k , or top- p) selects the next token.

1.4 Key Terminology

The following concepts are used throughout the paper and are defined here for clarity.

Token. The atomic unit of input/output. Roughly corresponds to one common word or sub-word fragment (approximately 0.75 natural-language words per token on average for English text).

Prefill phase. The initial stage of inference in which all tokens of the input prompt are processed in parallel to populate the KV cache. This phase is compute-bound.

Decode phase. The subsequent autoregressive loop in which new tokens are generated one at a time, each conditioned on all prior tokens. This phase is memory-bandwidth-bound.

KV cache. The stored key and value tensors for all past tokens across all layers, maintained to avoid recomputing attention for previously seen tokens on each decode step.

Quantisation. The process of representing model weights (and/or activations) at reduced numerical precision, e.g., replacing 32-bit or 16-bit floating-point values with 8-bit or 4-bit integers.

Pruning. Removing weights, attention heads, or entire layers from a trained model to reduce its parameter count and computational cost.

Knowledge Distillation. Training a smaller student model to reproduce the behaviour of a larger teacher model, transferring capability at reduced cost.

Throughput. The number of tokens generated per second across all concurrent requests; a system-level metric of primary importance for server-side deployment.

Latency. The time from the arrival of a request to the generation of the final token; a user-facing metric of primary importance for interactive and edge applications.

Edge device. Any computing platform with constrained memory, compute, and power budget that operates outside a data centre, including smartphones, embedded controllers, and automotive compute units.

1.5 Scope: Why Edge Devices?

While cloud-based inference remains the dominant deployment paradigm today, the trajectory of LLM adoption points strongly towards edge deployment. Smartphone vendors have begun integrating dedicated NPUs capable of running quantised 7B models [1]. Automotive and industrial systems increasingly require on-device reasoning without round-trip latency. Privacy regulations in healthcare and finance incentivise on-premise or on-device processing.

The edge deployment scenario is particularly demanding because it aggregates all four resource constraints simultaneously: memory limits prevent loading large models, bandwidth limits degrade decode throughput, power limits cap sustained performance, and compute limits restrict the complexity of techniques that can be applied at runtime. It

thus serves as the most informative stress test for efficiency techniques and is therefore the lens through which the methods surveyed in Chapter chapter 3 are evaluated.

2 The Inference Process and its Bottlenecks

This chapter traces the complete forward pass of a decoder-only Transformer from raw input text to a generated token, identifies the principal resource bottlenecks at each stage, and explains why those bottlenecks are not always confined to the inference phase alone. The presentation is intentionally concise but structured, as the bottlenecks identified here form the organising principle for the efficiency techniques surveyed in Chapter chapter 3.

2.1 Overview of the Inference Pipeline

Inference in a decoder-only Transformer proceeds in two temporally distinct phases that exhibit very different resource profiles.

Prefill phase. All n_{prompt} tokens of the input prompt are processed simultaneously in a single forward pass. Because all input tokens are available upfront, the attention computation is fully parallelised across the sequence dimension. This phase is *compute-bound*: the dominant cost is the matrix multiplications in the attention and FFN sub-layers, and the hardware’s arithmetic throughput is the limiting resource.

Decode phase. After the prefill, the model enters an autoregressive loop: at each step it processes a *single* new token, conditions on the full context (via the KV cache), and samples the next token. Because only one token is processed per step, the arithmetic intensity (ratio of floating-point operations to memory bytes accessed) drops by a factor of n . The dominant cost is now loading the P -parameter weight matrices and the accumulated KV cache from DRAM into the compute unit for every generated token. This phase is *memory-bandwidth-bound* [15].

The complete pipeline is illustrated in Figure figure 2.1.

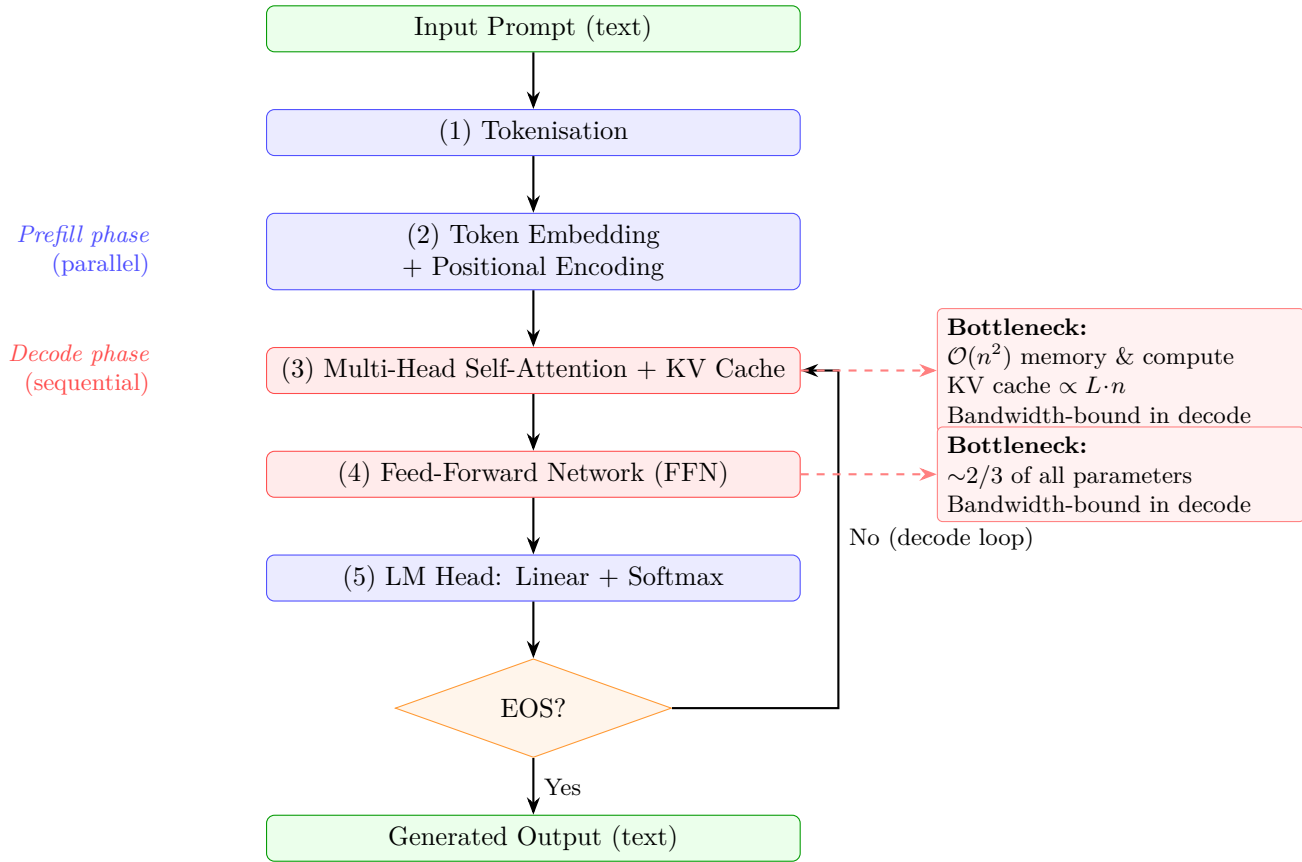


Figure 2.1: Schematic of the Transformer inference pipeline. Red-shaded steps (3, 4) represent the primary resource bottlenecks. The decode loop repeats steps 3–5 until an end-of-sequence token is produced.

2.2 Step-by-Step Bottleneck Analysis

2.2.1 Step 1: Tokenisation

The tokeniser maps a string of characters to a sequence of integer IDs using a fixed vocabulary and a pre-trained merge table (e.g., Byte-Pair Encoding). This step is computationally negligible. Its resource relevance is indirect: a poorly chosen tokenisation scheme may increase the average sequence length n , which in turn scales the KV cache and the cost of the attention computation.

2.2.2 Step 2: Token Embedding and Positional Encoding

The embedding lookup is a simple table indexing operation: $O(n \cdot d_{\text{model}})$ memory reads with no arithmetic. For large vocabularies (e.g., 128 000 tokens) the embedding matrix $\mathbf{W}_E$ can occupy several hundred megabytes. Because the embedding table is accessed irregularly (depending on which tokens appear in the prompt), it exercises memory bandwidth with poor cache locality.

2.2.3 Step 3: Multi-Head Self-Attention

The attention sub-layer is the most analytically interesting bottleneck. Its resource costs arise from two sources:

Arithmetic cost. Computing $\mathbf{QK}^\top$ for a sequence of length n requires $O(n^2 d_k)$ floating-point operations, giving quadratic growth in sequence length. During prefill this dominates for long contexts. During decode, $n_{\text{new}} = 1$, so the arithmetic cost collapses to $O(nd_k)$ —a single dot-product per head per past token.

KV cache memory. To avoid recomputing keys and values for every past token, the model caches them after prefill. As derived in Section section 1.2, this cache can equal or exceed the model weights in size for long sequences. Its growth with sequence length is the principal obstacle to serving many concurrent long-context requests, and to fitting the inference workload on an edge device.

2.2.4 Step 4: Feed-Forward Network

The FFN sub-layer applies two (or three, in gated architectures) large matrix multiplications per layer. For a standard configuration, $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times 4d_{\text{model}}}$ and $\mathbf{W}_2 \in \mathbb{R}^{4d_{\text{model}} \times d_{\text{model}}}$, so the FFN accounts for $8d_{\text{model}}^2$ parameters per layer—approximately twice the parameter count of the attention projection matrices. Across all L layers, the FFN weights represent roughly 65% of total model parameters.

During prefill, the FFN is compute-bound: the large matrix multiplications achieve high MFU. During decode (single-token input), the same weight matrices must be read from DRAM but perform only $O(d_{\text{model}})$ arithmetic operations per column—making this phase heavily bandwidth-bound.

2.2.5 Step 5: LM Head and Sampling

The LM head projects the final hidden state to a logit vector of size $|V|$. This is another large matrix multiplication ($\mathbb{R}^{d_{\text{model}} \times |V|}$, which is often shared with $\mathbf{W}_E$ to save memory). The subsequent softmax and sampling step are computationally inexpensive but introduce a sequential dependency: the next token cannot be determined until sampling is complete, enforcing a fundamental serialisation in standard autoregressive decoding.

2.3 Limitations Beyond Inference

Several of the bottlenecks described above are not exclusive to the inference phase. This distinction is relevant when evaluating optimisation strategies.

Model weight size. The memory footprint of the weights is identical whether the model is being trained or served. Compression techniques such as quantisation and pruning that reduce weight memory therefore benefit both phases. Their costs—accuracy degradation and potential increases in calibration compute—are also shared.

KV cache growth. The KV cache is an inference-time construct: it does not exist during training. However, its size is a direct consequence of architectural decisions made before training (number of layers L , number of attention heads h , head dimension d_k), which cannot be changed post hoc without retraining. Architectural choices such as Grouped-Query Attention (GQA) [2] or Multi-Query Attention (MQA) [16] that reduce the per-layer KV footprint must therefore be baked into the model from the outset.

Arithmetic intensity. The bandwidth-vs-compute characterisation of decode-phase inference stems from the small batch size of autoregressive generation (effective batch size $= 1$ per concurrent request). Techniques such as continuous batching, which increase the effective batch size by grouping multiple requests, can shift the operating point towards the compute-bound regime—but require system-level changes and are primarily applicable in server settings.

Table 2.1 summarises the main bottlenecks and their primary manifestations in each phase.

Table 2.1: Summary of inference bottlenecks by phase. n = sequence length, P = total parameters, L = layers, BW = memory bandwidth.

Component	Prefill	Decode	Primary resource	Scales with
Attention (arithmetic)	$O(n^2d)$	$O(nd)$	Compute / BW	n^2 / n
KV cache (memory)	$O(Lnd_k)$	$O(Lnd_k)$	Memory	n
FFN weights (BW)	Compute-bound	BW-bound	BW	P
LM head	Small	Small	BW	$ V $

3 Towards Efficient Inference: Techniques and Solutions

Drawing on the bottleneck analysis of Chapter chapter 2, this chapter systematically surveys the techniques that have been developed to reduce the resource demands of LLM inference. For each technique the discussion covers its core mechanism, the bottleneck it addresses, its accuracy–efficiency trade-off, and its practical implications for edge deployment. Figure figure 3.1 provides an overview of how the techniques presented in this chapter map to the bottlenecks identified in Chapter chapter 2.

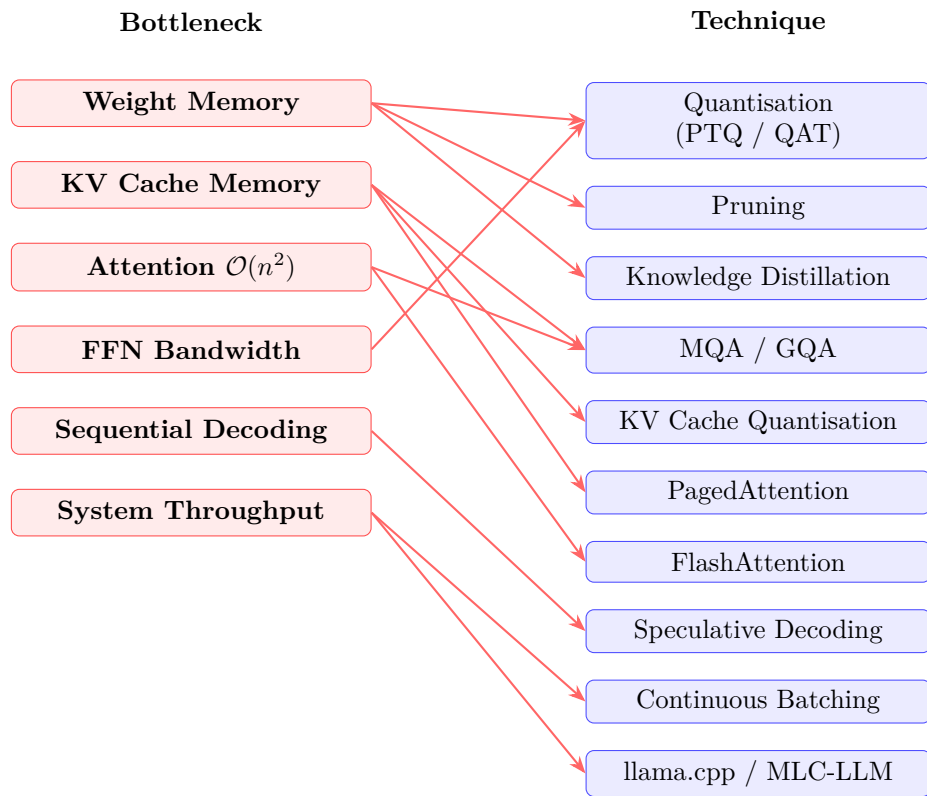


Figure 3.1: Mapping of inference bottlenecks (red) to the efficiency techniques (blue) surveyed in this chapter. Arrows indicate which technique targets which bottleneck; several techniques address multiple bottlenecks simultaneously.

3.1 Model Compression

Model compression techniques reduce the size of the model itself—shrinking weight memory and, consequently, the bandwidth required to load those weights during each decode step. They are pre-deployment operations and impose a one-time cost in compute or data for calibration.

3.1.1 Quantisation

Quantisation is the process of representing the floating-point weights (and optionally activations) of a trained model at reduced numerical precision. It is currently the most widely adopted compression technique for edge LLM deployment because it yields large memory reductions with acceptable quality loss and requires no architectural changes.

Why quantisation helps. A weight stored in FP16 occupies 2 bytes. Representing it instead in 8-bit integer format (8-bit Integer (INT8)) halves weight memory and doubles arithmetic throughput on hardware that natively supports INT8 matrix multiply (e.g., NVIDIA Tensor Cores, ARM Cortex-A78 NEON). Reducing to 4-bit integers (4-bit Integer (INT4)) achieves a $4\times$ compression relative to FP16, enabling a 7B model to fit in as little as 3.5 GB of memory.

Post-Training Quantisation (PTQ). In Post-Training Quantisation (PTQ), a fully trained model is quantised without further gradient-based updates. The simplest approach, round-to-nearest (RTN), maps each weight w to the nearest representable integer value; it is fast but introduces quantisation error that can significantly degrade perplexity below 4 bits.

More sophisticated PTQ methods achieve substantially better accuracy. **GPTQ** [8] applies a second-order optimisation using the inverse Hessian of the layer-wise reconstruction error to determine quantisation order and compensate remaining weights for the quantisation error of previously quantised weights. This allows accurate weight-only quantisation to INT4 with less than 1 point of perplexity increase on LLaMA-class models.

AWQ [13] takes a complementary perspective: it observes that the importance of a weight is correlated with the magnitude of the corresponding activation, and that protecting a small fraction (1%) of salient weights in higher precision (while quantising the rest to INT4) preserves most of the model quality. AWQ is *activation-aware* in the sense that it uses a small calibration dataset to identify these salient channels before quantisation.

LLM.int8() [7] addresses quantisation from the activation side. It observes that LLM

activations contain a small number of outlier features with magnitudes $100\times$ larger than the mean. Quantising these outliers causes disproportionate error. The solution is a mixed-precision decomposition: outlier feature dimensions are kept in FP16 while the remaining $\sim 99.9\%$ are multiplied in INT8, with no loss in model quality at INT8 precision on models above 6.7B parameters.

Quantisation-Aware Training (QAT). In Quantisation-Aware Training (QAT), the quantisation operation is simulated during training by inserting ‘fake quantisation’ nodes that round activations and weights during the forward pass but compute gradients through the rounding operation via the straight-through estimator. The model therefore learns to be robust to quantisation noise. QAT typically achieves better accuracy than PTQ at very low bit widths (2–4 bits), but requires re-training access to the original data and substantial compute—making it less practical when a model’s training data is proprietary.

Edge implications. Quantisation is the *de facto* entry point for edge LLM deployment. Tools such as llama.cpp (see Section section 3.4) implement custom 2–8 bit integer formats (GGUF quantisation) specifically optimised for ARM and x86 CPUs, enabling consumer hardware to run 7B and 13B models interactively. The dominant trade-off is quality vs. compression ratio: below 4 bits, perplexity degradation becomes increasingly noticeable, and the optimal bit-width is task-dependent [3].

3.1.2 Pruning

Pruning removes parameters from the model that contribute least to its predictions, reducing both weight memory and FLOP count.

Unstructured pruning sets individual weights to zero based on a criterion (most commonly magnitude: $|w_{ij}| < \theta$). High sparsity levels (80–90%) can be achieved with minimal quality loss for moderately sized models. However, irregular sparsity patterns yield little practical speedup on hardware without sparse compute support, because memory accesses to weight matrices remain irregular and prevent efficient vectorisation.

Structured pruning removes entire, regular sub-structures—attention heads, FFN rows/columns, or entire layers—producing a dense smaller model that achieves genuine speed improvements on standard hardware without requiring specialised sparse kernels.

LLM-Pruner [14] automates structured pruning of LLMs by analysing the dependencies between coupled weight groups (e.g., attention heads and their corresponding projection

matrices) and applying gradient-based importance estimation to select which groups to remove. After pruning, a brief post-training or Low-Rank Adaptation (LoRA)-based fine-tuning step recovers accuracy. In experiments on LLaMA, LLM-Pruner achieves a 20% parameter reduction with less than 3 points perplexity degradation and no access to the original pre-training data.

Head pruning specifically targets attention heads. Studies have shown that many attention heads learn redundant or near-uniform patterns and can be removed without quality loss [3]. Removing k heads out of h per layer reduces the attention KV footprint proportionally, providing a complementary benefit to techniques that reduce KV cache size.

Layer pruning removes entire Transformer blocks. Because blocks are applied sequentially, removing a block reduces latency proportionally. Empirical results on LLaMA-2 show that removing up to 25% of layers causes less than 5% perplexity increase on standard benchmarks; the degradation is larger for tasks requiring deep reasoning chains.

Edge implications. Structured pruning is particularly attractive for edge deployment because the resulting model is simply a smaller version of the original and requires no specialised runtime. A pruned-then-quantised 7B model can realistically be served on a high-end smartphone: pruning to a 5B effective parameter count followed by INT4 quantisation reduces memory from 14 GB to approximately 2.5 GB.

3.1.3 Knowledge Distillation

Knowledge distillation (KD) [10] trains a smaller *student* model to mimic the behaviour of a larger *teacher* model. Rather than training the student on hard labels (the ground-truth next token), the student is trained to match the teacher’s soft probability distribution over the vocabulary. Because the teacher’s output distribution encodes information about semantically related tokens—including near-misses and plausible alternatives—the student receives a richer training signal than hard-label supervision.

Task-specific distillation fine-tunes a small student on a task-specific corpus using the teacher’s outputs as supervision. This produces compact models optimised for narrow applications (e.g., a 1B model for a specific domain), but the student does not generalise beyond the training distribution.

Task-agnostic / general distillation trains a student from scratch (or from a pre-trained checkpoint) to mimic the pre-training distribution of the teacher. This is far more expensive (requiring access to large corpora and significant compute) but produces a student that retains general-purpose capabilities.

White-box vs. black-box distillation. When the teacher’s internal representations are accessible (*white-box*), the student can be trained to match not only the output logits but also intermediate hidden states and attention maps, providing additional supervision signal. When only the teacher’s output text is available (*black-box*), the student is trained on the teacher-generated text as a dataset.

Edge implications. Distillation is the only compression technique that does not start from a large model: it produces a small model from the outset. For edge deployment, 1B and 3B-parameter models distilled from 70B+ teachers (e.g., Phi-2, Phi-3 from Microsoft) have demonstrated that compact architectures can retain a large fraction of the teacher’s reasoning capability when the distillation corpus is carefully curated.

3.2 Inference-Level Optimisations

These techniques modify the attention mechanism or the KV cache management strategy without altering the model weights. They can be applied at inference time, often transparently to the model.

3.2.1 Multi-Query and Grouped-Query Attention

Standard MHA maintains one independent set of key and value projection matrices per head. With h heads, the KV cache therefore stores h distinct key and value tensors per layer and per token, with total per-token cache size $2Lhd_k$.

MQA [16] proposes that all query heads share a single key head and a single value head. The KV cache therefore collapses to $2L \cdot 1 \cdot d_k$ per token—a factor-of- h reduction. Because the KV cache is the second largest contributor to memory consumption after model weights, this reduction is highly significant at edge scale. The trade-off is a slight decrease in model quality (approximately 0.5–1% on downstream benchmarks), which is generally acceptable [16].

GQA [2] generalises MQA by grouping the h query heads into g groups, each sharing one key/value head. With $g = h$ this reduces to standard MHA; with $g = 1$ to MQA.

In practice, $g = h/4$ to $h/8$ recovers most of the MHA quality while approaching MQA efficiency. GQA has been adopted in LLaMA-2 (70B), LLaMA-3, and Mistral, indicating that the field has largely converged on this as the standard attention variant.

Edge implications. Since MQA/GQA must be incorporated during pre-training, they are not applicable to already-trained models. However, GQA can be retrofitted via a brief fine-tuning step (uptraining from MHA checkpoints) at a fraction of the original training cost [2]. For new models targeting edge deployment, GQA should be considered mandatory.

3.2.2 FlashAttention

The standard MHA implementation loads the full $n \times n$ attention score matrix into HBM before applying the softmax and value aggregation. For a sequence length of $n = 4096$ tokens with FP16 precision, this matrix occupies $n^2 \times 2$ bytes = 32 MB per head. With 32 heads, this is 1 GB of temporary HBM traffic per layer—a dominant bandwidth cost.

FlashAttention [6] reformulates the attention computation to avoid materialising the full attention matrix. It tiles the input matrices into blocks that fit entirely in the GPU’s fast on-chip SRAM, and fuses the computation of $\mathbf{QK}^\top$, softmax, and $\text{softmax}(\cdot)\mathbf{V}$ into a single kernel. This eliminates the round-trip to HBM for the intermediate attention matrix, reducing HBM accesses from $O(n^2)$ to $O(n)$ while computing the numerically identical result (it is an exact, not approximate, algorithm).

Practically, FlashAttention achieves 2–4 $\times$ speedup on attention computation relative to the naive implementation, with proportionally lower energy consumption—making it directly beneficial for edge inference on any hardware with a fast cache hierarchy.

3.2.3 KV Cache Optimisation

PagedAttention. The KV cache grows dynamically with sequence length. In the standard contiguous allocation scheme, each new request pre-allocates a fixed-size maximum-length block of KV memory, leading to severe internal fragmentation when requests terminate early. Across thousands of concurrent requests on a server, this wastage can reduce effective throughput by 60–80% [11].

PagedAttention [11], implemented in the vLLM framework, adapts the virtual memory paging concept to the KV cache. Each request’s KV vectors are stored in fixed-size *pages* that need not be physically contiguous. A page table maps logical to physical page

addresses at inference time. This eliminates nearly all internal and external fragmentation, increasing the effective number of concurrent requests that can be served from the same GPU memory by 2–4 $\times$.

KV cache quantisation. The KV cache can itself be stored at reduced precision. Quantising the cached key/value tensors to INT8 halves cache memory with minimal perplexity impact. More aggressive quantisation to INT4 is possible with careful calibration, providing a further 2 $\times$ reduction.

KV cache eviction. For very long sequences, it is possible to *evict* (discard) cache entries for tokens that are unlikely to be attended to by future queries, based on heuristics such as attention score recency or magnitude. StreamingLLM and similar methods maintain a fixed-size cache window, enabling effectively unbounded context length at the cost of some context fidelity.

3.3 Decoding Strategies

3.3.1 Speculative Decoding

Standard autoregressive decoding generates one token per forward pass, enforcing a strict latency lower bound of (number of generated tokens) $\times$ (time per forward pass). This is wasteful when the full model is large but most token generations are ‘easy’ (highly predictable).

Speculative decoding [12] exploits this insight with a two-model approach:

1. A small, fast *draft model* (e.g., 1–3B parameters) generates K candidate tokens autoregressively in K forward passes.
2. The large *target model* processes all K draft tokens in a single forward pass (which is parallelisable, like prefill) and accepts or rejects each draft token using a rejection sampling criterion that guarantees the accepted sequence has the same distribution as the target model’s output.

When the draft model’s predictions match the target with probability α (the *acceptance rate*), the expected number of tokens produced per target-model forward pass is $\frac{1-\alpha^{K+1}}{1-\alpha}$. For $\alpha \approx 0.8$ and $K = 5$, this yields approximately 3.5 tokens per call to the target model—a 3.5 $\times$ latency improvement with no loss in output quality [12].

For edge deployment, the ‘large model’ and ‘draft model’ may themselves be a quantised medium model and a tiny model respectively. The framework is flexible: any two models with matching tokenisers and vocabulary can be paired.

3.3.2 Continuous Batching

Traditional static batching waits until a fixed number of requests arrive and then processes them together. Requests of different lengths must be padded to a common length (wasting compute) and the entire batch must finish before new requests are admitted (increasing average latency).

Continuous (or iteration-level) batching inserts new requests into the running batch at the earliest available slot (e.g., whenever a request produces an EOS token and vacates its slot). This keeps hardware utilisation high and reduces median latency significantly. Combined with PagedAttention, continuous batching enables high-throughput serving that is relevant for cloud deployments and, increasingly, for shared edge inference servers in enterprise settings.

3.4 System-Level Runtimes and Edge Frameworks

Efficiency techniques are only realised in practice when they are implemented in well-engineered software runtimes that expose hardware capabilities efficiently. Several frameworks have emerged specifically for LLM inference, with differing strengths on server and edge hardware.

3.4.1 llama.cpp

`llama.cpp` [9] is a C/C++ inference runtime designed for deployment on commodity hardware, including ARM CPUs (Apple Silicon, Android), x86 CPUs, and consumer GPUs. Its key design decisions include:

- **GGUF quantisation format:** a family of mixed-precision weight quantisation schemes (2–8 bits per weight) with hand-tuned dequantisation kernels for NEON (ARM SIMD), AVX2, and CUDA.
- **Minimal dependencies:** the library has no deep learning framework dependency, making it straightforward to compile and deploy on embedded targets.

- **Metal / Vulkan backend:** hardware-accelerated compute on Apple Neural Engine and Mali/Adreno GPUs on mobile devices.

In practice, llama.cpp enables a quantised (Q4_K_M) 7B model to generate tokens at 20–40 tokens/second on a modern laptop CPU and at 60–80 tokens/second on an Apple M-series chip—sufficient for interactive use [1].

3.4.2 MLC-LLM

MLC-LLM [5] (Machine Learning Compilation (MLC) Large Language Model) takes a compilation-based approach. It uses the Apache TVM deep learning compiler to generate hardware-specific compute kernels from a high-level model description, targeting iOS (Metal), Android (OpenCL, Vulkan), WebGPU, CUDA, and ROCm from a single code base. Key advantages include:

- Auto-tuning of tiling and scheduling parameters for the target hardware.
- Support for both weight quantisation and KV cache quantisation.
- On-device inference on smartphones (e.g., LLaMA-2 7B at 3–8 tokens/second on a Pixel 7 Pro).

The compilation step adds several minutes of one-time overhead but produces deployment artefacts that run efficiently with no runtime compilation cost.

3.4.3 vLLM

vLLM [11] is optimised for *server-side* high-throughput serving. Its core contribution is PagedAttention (Section section 3.2.3), which dramatically reduces KV cache memory fragmentation. vLLM additionally implements continuous batching, FlashAttention, and quantised inference, providing a production-grade serving stack for data-centre deployment. While not directly targeted at edge hardware, vLLM establishes the baseline for cloud-side efficient inference from which edge-focused frameworks draw inspiration.

3.4.4 Comparison and Edge Suitability

Table 3.1 compares the three frameworks across the dimensions most relevant to edge deployment.

Table 3.1: Comparison of selected inference frameworks for edge deployment. ✓ = supported, ~ = partial support, × = not supported.

Framework	Edge HW	Mobile	Quantisation	Throughput focus
llama.cpp	✓	✓	✓ (2–8 bit)	✓
MLC-LLM	✓	✓	✓ (4–8 bit)	~
vLLM	×	×	~ (INT8)	✓

3.5 Discussion: Trade-offs and Edge-Specific Considerations

No single technique dominates across all deployment scenarios. The efficiency gains and quality losses described above interact in ways that must be understood holistically when targeting edge hardware.

3.5.1 The Efficiency–Accuracy Trade-off

Compression techniques invariably introduce some quality degradation. The magnitude of this degradation depends on:

- **Bit width:** quantisation from FP16 to INT8 is nearly lossless; from INT8 to INT4 introduces a noticeable but usually acceptable quality reduction; below INT4, degradation accelerates [8].
- **Task sensitivity:** arithmetic, logical reasoning, and mathematical tasks degrade faster under quantisation than open-ended text generation [3].
- **Model scale:** larger models exhibit greater robustness to compression. A 70B model quantised to INT4 often outperforms an uncompressed 7B model, making the question ‘which compressed large model vs. which uncompressed small model’ a central practical question for edge deployment.

3.5.2 Orthogonality and Composability

Many of the techniques described in this chapter are *orthogonal*: they address different bottlenecks and can be applied simultaneously. A practical edge stack might combine:

1. **Distillation** (at training time) to obtain a smaller base model.
2. **GQA** (in the model architecture) to reduce KV cache size.

3. **GPTQ or AWQ quantisation** (post-training) to compress weights to INT4.
4. **FlashAttention** (at inference time) to reduce attention bandwidth.
5. **Speculative decoding** (at inference time) to improve throughput.
6. **llama.cpp or MLC-LLM** as the runtime to execute efficiently on ARM/mobile hardware.

Applied together, these techniques have enabled 7B-class models to be deployed interactively on smartphones in 2024—an outcome that would have seemed impractical just two years earlier.

3.5.3 The Fundamental Edge Constraint: Memory

Across all dimensions, memory remains the hardest constraint on edge devices. Compute and energy can often be traded against latency by reducing clock frequency; bandwidth can be partially mitigated by caching. But there is no recourse when a model simply does not fit in the available DRAM: inference is not possible. This makes weight quantisation and architectural choices that reduce the KV cache (such as GQA) the most impactful interventions for edge deployment, not as performance optimisations but as enabling technologies [1].

4 Conclusion

This paper has traced the resource challenges of LLM inference from first principles through to practical deployment solutions, with a sustained focus on edge hardware as the most demanding deployment target.

Chapter chapter 1 established that the four resource dimensions of LLM inference—compute, memory, memory bandwidth, and energy—are not equally constraining in all settings. On edge devices, memory capacity is the hard, non-negotiable limit: a model that does not fit in DRAM simply cannot run. Memory bandwidth is the dominant performance bottleneck during the decode phase, and energy determines the sustained throughput a device can deliver over time.

Chapter chapter 2 analysed the inference pipeline step by step and identified its two most resource-intensive components: the self-attention sub-layer, which incurs $O(n^2)$ compute during prefill and a growing KV cache during decode, and the FFN sub-layer, which accounts for roughly two-thirds of all model parameters and is memory-bandwidth-bound during decode. The chapter also clarified that while the KV cache is an inference-specific structure, its size is determined by architectural choices made at training time—meaning that the efficiency of inference is, in part, a function of how thoughtfully the model was designed.

Chapter chapter 3 surveyed the state-of-the-art techniques for addressing these bottlenecks. Several key findings emerge from this analysis.

4.1 Key Findings

Quantisation is the single most impactful intervention for edge deployment. Reducing weight precision from FP16 to INT4 achieves a $4\times$ compression of model weights, making 7B-class models feasible on consumer hardware with 4–8 GB of memory. Methods such as GPTQ [8] and AWQ [13] demonstrate that this compression can be achieved with minimal perplexity degradation, particularly for models above 7B parameters.

GQA is now essential architecture. The reduction of the KV cache footprint by a factor of h/g (where h is the number of query heads and g is the number of key-value

head groups) is one of the most memory-efficient interventions available. It is cost-free in a newly trained model and recoverable at low cost through uptraining. Given the disproportionate role of KV cache in edge memory budgets, GQA should be considered a baseline requirement for any model targeting edge deployment.

FlashAttention makes long-context inference practical. By eliminating the quadratic memory traffic of the attention matrix, FlashAttention [6] substantially reduces both latency and energy consumption for prefill, and makes longer context windows feasible without exceeding on-chip memory limits.

Speculative decoding provides latency improvements without quality loss. For interactive applications—which are the primary use case for edge inference—reducing latency per generated token is more important than maximising throughput. Speculative decoding [12] directly targets this metric and is composable with other techniques.

Dedicated edge runtimes bridge the gap between algorithmic efficiency and hardware performance. llama.cpp [9] and MLC-LLM [5] demonstrate that practical edge inference is achievable with current 7B-class models when quantisation, hardware-specific SIMD kernels, and efficient memory management are integrated in a single, well-engineered stack.

4.2 Trade-off Summary

No combination of techniques eliminates all resource constraints. The residual trade-offs that practitioners must navigate include:

- **Compression depth vs. task quality.** Below INT4 precision, perplexity degradation becomes task-dependent and can be severe for arithmetic and logical reasoning. The optimal compression level is application-specific and must be validated empirically.
- **Model size vs. instruction-following capability.** Very small models (1–3B parameters), even after distillation, exhibit limited complex instruction following. Practical edge deployment of capable assistants currently requires 7B+ parameters, constraining the minimum hardware target.
- **Speculative decoding overhead.** The draft model adds memory and compute overhead. On heavily memory-constrained devices, the additional memory for the draft model may not be available, limiting speculative decoding to devices with at least 6–8 GB of available memory.

- **Static compression vs. dynamic efficiency.** Model compression (quantisation, pruning) operates on the model and is applied once; inference-level optimisations (FlashAttention, speculative decoding) operate at runtime. The two categories are complementary but require integrated engineering effort to combine reliably.

4.3 Future Directions

Several open problems define the frontier of resource-efficient edge inference.

Sub-4-bit quantisation. Reliable INT2 and INT3 quantisation methods that preserve task quality would reduce the memory floor for a 7B model to below 2 GB, opening the majority of current smartphones as deployment targets. Emerging methods that combine low-bit weight quantisation with full-precision activations show promise in this direction.

Hardware–software co-design. Current edge NPUs are optimised for vision models (fixed-shape convolutions) rather than the variable-sequence, attention-heavy workloads of LLMs. Purpose-designed LLM accelerators with on-chip KV cache storage and native support for irregular sparse patterns could unlock an order-of-magnitude efficiency improvement beyond what software alone can achieve.

Adaptive inference. Fixed-capacity models always over-provision compute for simple queries. Adaptive or early-exit architectures that route ‘easy’ queries to shallow layers (or a small model) and ‘hard’ queries to deeper layers (or a large model) could significantly reduce average energy consumption without quality loss on the overall query distribution.

On-device fine-tuning and personalisation. Efficient LoRA-based fine-tuning methods are already small enough to run on high-end smartphones. Integrating personalisation into the edge inference loop—without degrading privacy or exhausting the battery—remains an open engineering and research challenge.

The convergence of quantisation, architectural improvements (GQA, FlashAttention), and purpose-built edge runtimes has already made interactive on-device LLM inference a practical reality for a subset of hardware. The trajectory of the field suggests that within the next two to three years, capable language models will be routinely deployable across the full range of consumer edge devices.

Bibliography

- [1] “A review on edge large language models: Design, execution, and applications,” *ACM Computing Surveys*, 2024. [Online]. Available: <https://dl.acm.org/doi/10.1145/3719664>
- [2] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “GQA: Training generalised multi-query transformer models from multi-head checkpoints,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.13245>
- [3] G. Bai, Z. Chai, C. Ling, S. Wang, J. Lu, C. Li, and L. Yao, “Beyond efficiency: A systematic survey of resource-efficient large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.00625>
- [4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [5] T. Chen *et al.*, “MLC-LLM: Universal LLM deployment engine with ML compilation,” GitHub repository, 2023. [Online]. Available: <https://github.com/mlc-ai/mlc-llm>
- [6] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: <https://arxiv.org/abs/2205.14135>
- [7] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: <https://arxiv.org/abs/2208.07339>
- [8] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” 2023. [Online]. Available: <https://arxiv.org/abs/2210.17323>

- [9] G. Gerganov and contributors, “llama.cpp: C/C++ port of Meta’s LLaMA model,” GitHub repository, 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [10] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” 2015. [Online]. Available: <https://arxiv.org/abs/1503.02531>
- [11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [12] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023. [Online]. Available: <https://arxiv.org/abs/2211.17192>
- [13] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proceedings of Machine Learning and Systems*, vol. 6, 2024. [Online]. Available: <https://arxiv.org/abs/2306.00978>
- [14] X. Ma, G. Fang, and X. Wang, “LLM-Pruner: On the structural pruning of large language models,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2305.11627>
- [15] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Young, and J. Dean, “Efficiently scaling transformer inference,” in *Proceedings of Machine Learning and Systems*, vol. 5, 2023. [Online]. Available: <https://arxiv.org/abs/2211.05102>
- [16] N. Shazeer, “Fast transformer decoding: One write-head is all you need,” 2019. [Online]. Available: <https://arxiv.org/abs/1911.02150>
- [17] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “LLaMA: Open and efficient foundation language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.13971>
- [18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>